

SBA Loan Default Modeling

Business decision, feature engineering, validation workflow, model theory, and current teaching-sample results

Team teaching deck

How I want you to use this deck

- First, understand the approval decision and the profit rule.
- Then, understand why we created the features before comparing models.
- After that, each model follows the same pattern: theory, code logic, validation result.
- When you run your assigned model, keep the same scoreboard so our results are comparable.

Main idea

The model is only useful when it becomes a clear approve/deny policy.

Part	What we cover
1	Business rule and target
2	Leakage audit and feature engineering
3	Split, preprocessing, helper functions
4	Model families: theory, code, result
5	Leaderboard, profit curve, ROC/calibration, error analysis
6	What the team should do next

I am keeping the flow close to the Python teaching file, so you can move from slides to code without guessing where things are.

The business decision

For every loan application, we need one decision:

loan profile $\rightarrow P(\text{default}) \rightarrow$ approve or deny

A normal classifier asks whether the label was predicted correctly. Here we also need to ask whether the approval decision made money.

Decision	Actual outcome	Business effect
Approve	paid in full	earn profit
Approve	default	lose money
Deny	paid in full	miss profit
Deny	default	avoid loss

Target variable and payoff rule

Target

$$y = \begin{cases} 1, & \text{charged off / default} \\ 0, & \text{paid in full} \end{cases}$$

MIS_Status = CHGOFF becomes 1.

MIS_Status = P I F becomes 0.

Profit if loan amount is D

Case	Profit
approve + paid	$+0.05D$
approve + default	$-0.25D$
deny	0

D is DisbursementGross.

Break-even threshold

Let $p = P(\text{default})$ and $D = \text{loan amount}$.

$$\begin{aligned} E(\text{profit from approval}) &= (1 - p)(0.05D) + p(-0.25D) \\ &= 0.05D - 0.30Dp \end{aligned}$$

Approval breaks even when:

$$0.05D - 0.30Dp = 0 \quad \Rightarrow \quad p = \frac{1}{6} \approx 16.67\%$$

Main idea

The theoretical rule is: approve if $P(\text{default}) \leq 1/6$. We still check validation profit because model probabilities are not always perfectly calibrated.

Why accuracy is not enough

	Predict default / deny	Predict paid / approve
Actual default	caught bad loan	dangerous approval
Actual paid	denied good loan	earned profit

A denied good loan is an opportunity cost. An approved default creates an actual loss.

Validation baseline

On the teaching sample, approve-all earns about \$30.46M on validation. Any useful model should beat that baseline.

Leakage audit

Before modeling, I removed columns that would not be known at approval time.

Column	Why it leaks	Action
MIS_Status	it is the target	remove from X
ChgOffDate	known after default	remove
ChgOffPrinGr	charged-off amount	remove
BalanceGross	post-loan status	remove
ID/text fields	not useful here	remove

DisbursementGross is used for profit calculation. It is not used as a predictor in the default version of the file.

Feature engineering: why we did it

Feature engineering turns raw SBA fields into business signals.

Raw fields	Engineered feature	What it captures
SBA_Appv, GrAppv	Portion	SBA guarantee share
CreateJob, RetainedJob	jobs_total	promised job impact
jobs_total, GrAppv	jobs_per_dollar	impact relative to loan size
State, BankState	same_state_bank	local lender signal
Term	RealEstate	long-term loan proxy
NAICS	NAICS_sector	broad industry group

Feature formulas and variable meanings

$$\text{Portion} = \frac{\text{SBA_Appv}}{\text{GrAppv}}$$

$$\text{unguaranteed ratio} = 1 - \text{Portion}$$

$$\text{jobs total} = \text{CreateJob} + \text{RetainedJob}$$

$$\text{jobs per dollar} = \frac{\text{jobs total}}{\text{GrAppv}}$$

- SBA_Appv: guaranteed dollar amount.
- GrAppv: gross approved loan amount.
- Term: loan maturity in months.
- RealEstate=1 when Term $\geq$ 240.
- NAICS_sector: first two digits of NAICS.

Recession and interaction features

Recession exposure

A loan is marked as recession-exposed if its active period overlaps the Great Recession window.

$\text{DisbursementDate} \leq \text{recession end}$ and $\text{maturity date} \geq \text{recession start}$

Interactions used in the file

$\text{RealEstate} \times \text{Portion}$, $\text{Recession} \times \text{Portion}$, $\text{Recession} \times \text{RealEstate}$

Tree models can discover interactions through splits. Logistic regression needs interaction terms written into the feature set.

Feature engineering code logic

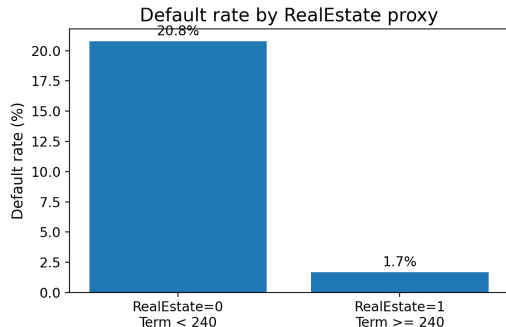
```
df_model["Portion"] = df_model["SBA_Appv"] / df_model["GrAppv"]
df_model["Portion"] = df_model["Portion"].replace([np.inf, -np.inf], np.nan).fillna(0)
df_model["unguaranteed_ratio"] = 1 - df_model["Portion"]
df_model["unguaranteed_amount"] = df_model["GrAppv"] - df_model["SBA_Appv"]

df_model["jobs_total"] = df_model["CreateJob"] + df_model["RetainedJob"]
df_model["jobs_per_dollar"] = df_model["jobs_total"] / df_model["GrAppv"]

df_model["same_state_bank"] = np.where(df_model["State"] == df_model["BankState"], 1, 0)
df_model["RealEstate"] = np.where(df_model["Term"] >= 240, 1, 0)
df_model["NAICS_sector"] = df_model["NAICS"].astype(str).str.zfill(6).str[:2]
```

EDA before modeling

Before training, I checked whether the engineered features line up with default behavior.



Feature group	Pattern in sample
RealEstate proxy	much lower default rate
RevLineCr flag	Yes/Unknown riskier than No
Urban/Rural	category differences are visible
NAICS sector	some sectors are riskier
Term	strong risk segmentation

Split before fitting transformations

The teaching file uses:

60% train, 20% validation, 20% test

Set	Used for	Should it tune final result?
Train	fit models and preprocessing	yes
Validation	choose models and thresholds	yes
Test	one final check	no tuning after this

Main idea

Validation and test data should not teach the imputer, scaler, or dummy-column structure.

Two preprocessors

Preprocessor	Used for	Why
<code>preprocess_scaled</code>	KNN, logistic, NN, LDA/QDA	scaling makes these models work
<code>preprocess_tree</code>	tree, bagging, RF, boosting	trees do not need scaling

Pipeline pattern

```
pipe = Pipeline(steps=[
    ("preprocess", preprocess_scaled),
    ("model", model)
])

pipe.fit(X_train, y_train)
prob_default = pipe.predict_proba(X_valid)[: , 1]
```

The pipeline prevents copy-paste mistakes. Each model gets the same train-only imputation, encoding, and scaling logic. For tree models, replace `preprocess_scaled` with `preprocess_tree`.

Appendix visual: KNN nearest-neighbor voting

KNN estimates default risk by looking at the most similar historical loans.

Choose K nearest loans
from the training set

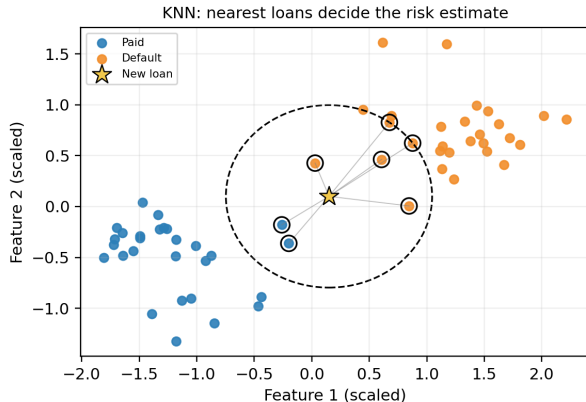


Average their outcomes
to estimate $P(\text{default})$

Formula

$P(\text{default} \mid x_0) = \text{average } y \text{ among nearest neighbors}$

Scaling matters because distance is the model logic.



Model map

Family	Model	Main idea
Similarity	KNN	compare to nearby historical loans
Tree	decision tree	split loans into risk segments
Ensemble	bagging/RF/boosting	combine many trees
Linear probability	logistic	linear log-odds score
Distribution	LDA/QDA	class distributions and Bayes rule
Neural network	MLP	hidden nonlinear risk signals

KNN: theory, code, result

For a new loan x_0 :

$$\hat{P}(Y = 1|X = x_0) = \frac{1}{K} \sum_{i \in N_K(x_0)} y_i$$

x_0 : new loan profile.

$Y = 1$: default.

$N_K(x_0)$: K nearest training loans.

K : number of neighbors. Teaching sample result:

AUC 0.860, profit \$53.56M.

```
knn = KNeighborsClassifier(  
    n_neighbors=51,  
    weights="distance",  
    p=2  
)  
fit_predict_evaluate(  
    "KNN", knn,  
    preprocess_obj=preprocess_scaled  
)
```

KNN beats approve-all, but distance is less stable after 158 encoded features.

Helper functions are just the scoreboard

These are the process that repeat throughout our work, so I turn them into functions

Function	Job
<code>loan_profit_vector</code>	profit or loss for each loan
<code>evaluate_prob_model</code>	metrics and profit at one threshold
<code>tune_threshold_by_profit</code>	searches thresholds on validation
<code>fit_predict_evaluate</code>	fits model, predicts, evaluates, returns tables

Every model outputs $P(\text{default})$. The functions turn that into the same validation scoreboard.

Decision tree: theory, code, result

A tree asks yes/no questions and sends each loan to a leaf.

$$\hat{P}(\text{default in leaf}) = \frac{\# \text{ default loans in leaf}}{\# \text{ loans in leaf}}$$

Gini impurity for default proportion p :

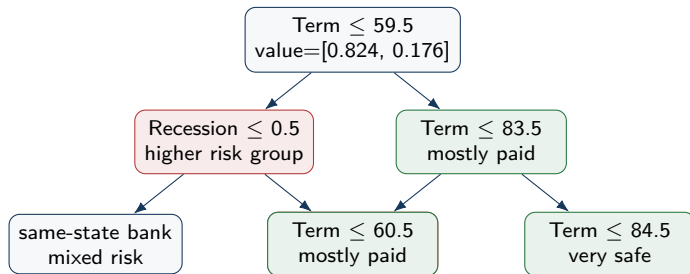
$$G = 2p(1 - p)$$

Lower Gini means the node is cleaner.

```
tree = DecisionTreeClassifier(  
    max_depth=6,  
    min_samples_leaf=500  
)  
fit_predict_evaluate(  
    "Decision Tree", tree,  
    preprocess_obj=preprocess_tree  
)
```

Teaching sample result: AUC 0.924, profit \$59.40M.

Teaching tree visualization



For example, take a look at this simple tree to understand how it works.

Bagging: theory, code, result

Bagging trains many trees on bootstrap samples and averages them.

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$$

B : number of trees.

$\hat{f}_b(x)$: prediction from tree b .

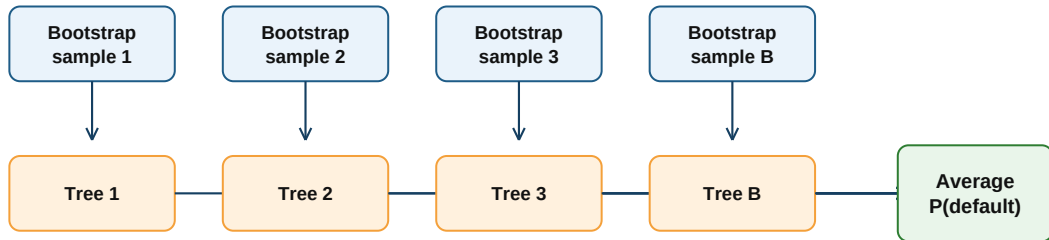
Bootstrap sample: rows sampled with replacement.

```
bagging = BaggingClassifier(  
    estimator=base_tree,  
    n_estimators=30,  
    max_samples=0.7,  
    bootstrap=True  
)  
fit_predict_evaluate(  
    "Bagging", bagging,  
    preprocess_obj=preprocess_tree  
)
```

Result: AUC 0.950, profit \$64.56M.

Appendix visual: Bagging averages unstable trees

A single tree can change a lot when the training rows change. Bagging trains many trees on bootstrap samples, then averages their predicted probabilities.



Prediction

bagged $P(\text{default})$ = average of tree probabilities

The value comes from stability: many noisy trees become one steadier estimate.

Random forest: theory and tuning lesson

Random forest starts from bagging, then adds feature randomness at each split.

Usually, feature randomness helps because trees become less similar. In this SBA sample, Term is very strong. If `max_features="sqrt"`, each split only sees around 12 of about 158 encoded features, so important features can be hidden.

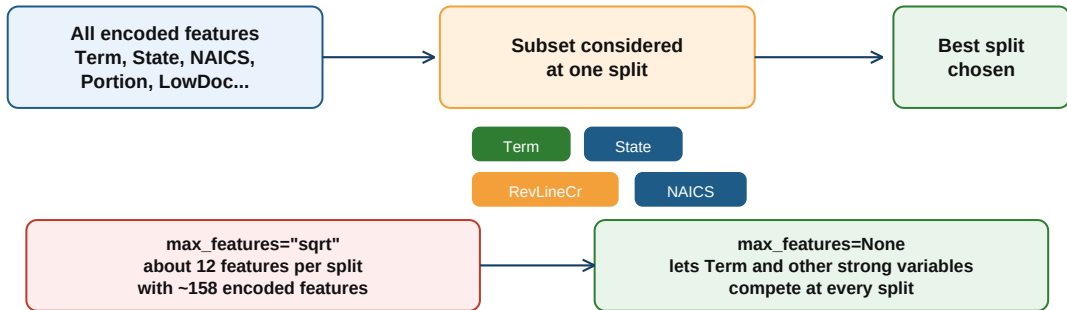
```
rf = RandomForestClassifier(  
    n_estimators=100,  
    max_depth=16,  
    min_samples_leaf=50,  
    max_features=None  
)
```

In the code file, you can run CV at the end.

CV confirmed that `max_features=None` worked best in this teaching sample.

Appendix visual: Random forest feature sampling

Random forest still uses many bootstrapped trees. The extra idea is feature sampling at each split.

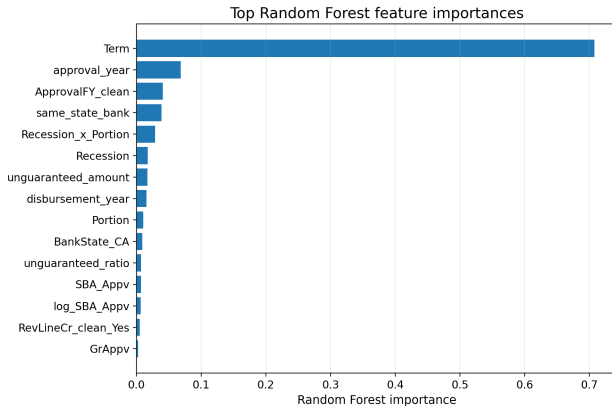


In our CV demo, `max_features=None` worked best. That is a tuning result for this dataset, not a universal rule.

Random forest: result and feature importance

Metric	RF tuned
AUC	0.963
Brier	0.054
Accuracy	0.892
Approval rate	74.8%
Approved default rate	2.1%
Net profit	\$67.78M

The model approves more loans than bagging while keeping default leakage low.

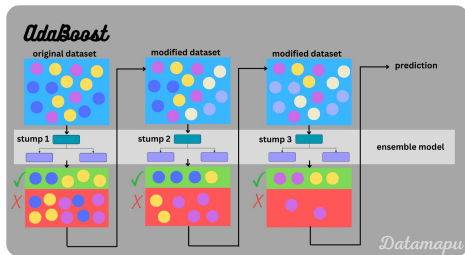


AdaBoost: theory, code, result

Boosting builds trees sequentially. Each new tree gives more attention to cases handled poorly before. AdaBoost starts with equal row weights, then increases the weights of misclassified observations. In this run, AdaBoost is useful, but it does not beat the stronger tree ensembles.

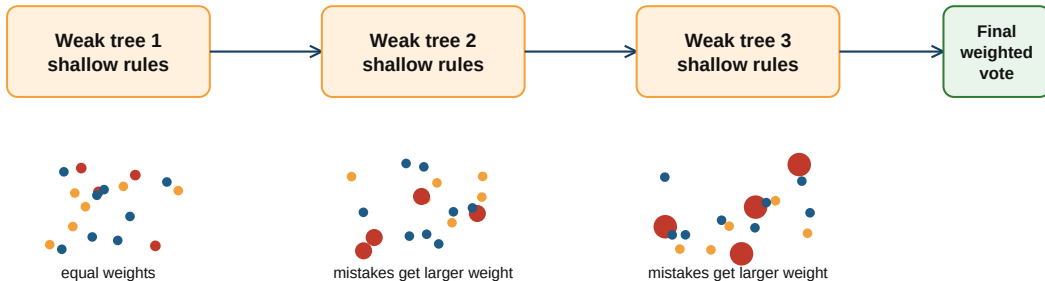
```
boosting = AdaBoostClassifier(  
    estimator=DecisionTreeClassifier(  
        max_depth=2,  
        min_samples_leaf=200),  
    n_estimators=100,  
    learning_rate=0.05  
)
```

Result: AUC 0.905, profit \$55.09M.



Appendix visual: Boosting focuses on mistakes

Boosting trains trees in sequence. After each weak tree, the next tree pays more attention to loans the previous trees handled poorly.



Bagging averages independent trees. Boosting adds weak trees one after another.

Logistic regression: theory, code, result

Logistic regression models log-odds:

$$\log \left(\frac{p(x)}{1 - p(x)} \right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

$p(x) = P(Y = 1|X = x)$: default probability.

β_j : coefficient for feature x_j .

C : inverse regularization strength in sklearn.

```
LogisticRegression(  
    penalty="l1",    # or l2, elasticnet  
    C=0.5,  
    solver="saga",  
    max_iter=3000  
)
```

Best logistic result here: Ridge, AUC 0.840,
profit \$51.36M.

Ridge, Lasso, ElasticNet

Model	Penalty	Effect
Ridge	$\lambda \sum_j \beta_j^2$	shrinks coefficients smoothly
Lasso	$\lambda \sum_j \beta_j $	can set coefficients to zero
ElasticNet	mix of L1 and L2	shrinkage plus selection

In the teaching sample, the logistic family is stable but weaker than tree ensembles. The issue is mostly model form: logistics regression cares more about classification, it does not care about profit optimization

LDA and QDA

Discriminant analysis compares class distributions.

$$P(Y = k|X = x) = \frac{P(X = x|Y = k)P(Y = k)}{P(X = x)}$$

$Y = k$: class, paid or default.

$X = x$: observed loan profile.

LDA uses shared covariance. QDA uses separate covariance by class.

Model	AUC	Profit
LDA	0.815	\$49.46M
QDA reg=0.1	0.801	\$45.12M

QDA

is unstable with many encoded columns,
so regularization matters.

Neural network: theory, code, result

A hidden unit learns:

$$h_j = g \left(w_{j0} + \sum_{m=1}^p w_{jm} x_m \right)$$

x_m : input feature.

w_{jm} : weight from feature m .

w_{j0} : bias.

$g(\cdot)$: activation such as ReLU.

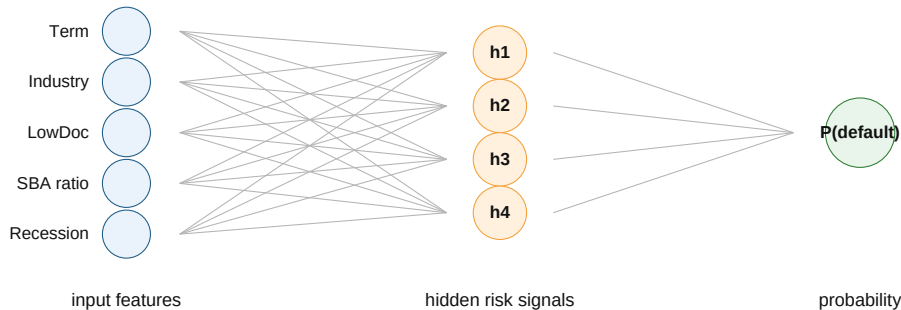
h_j : learned hidden risk signal.

```
MLPClassifier(  
    hidden_layer_sizes=(64,32),  
    activation="relu",  
    early_stopping=True,  
    max_iter=100  
)
```

Result: AUC 0.902, profit \$58.50M.

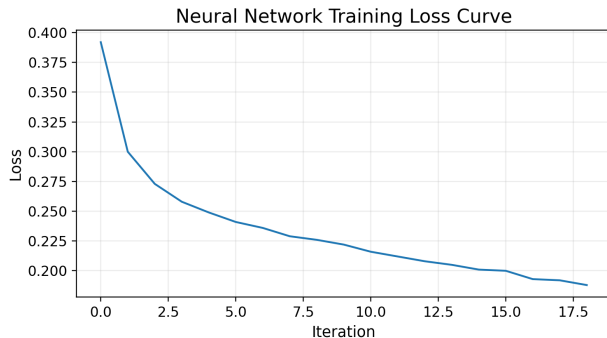
Appendix visual: Neural network hidden risk signals

The neural network combines inputs into hidden units. Each hidden unit can learn a different risk signal, then the output layer turns those signals into $P(\text{default})$.



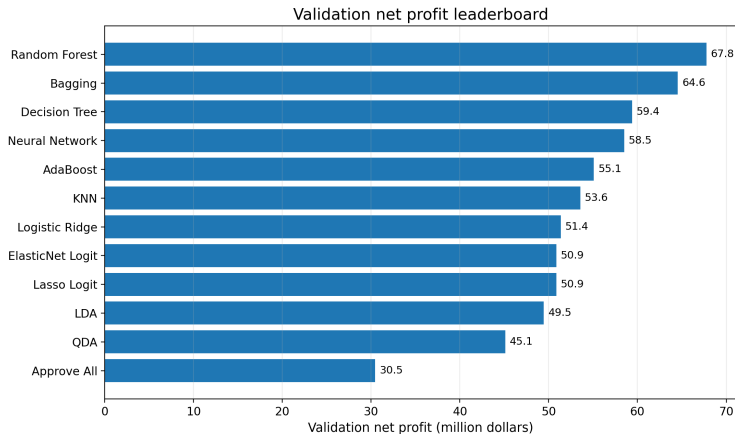
Hidden unit idea: $h = \text{activation}(\text{weighted sum of inputs} + \text{bias})$

Neural network training check



The loss falls steadily, so the model is learning. Early stopping stopped after 19 iterations in our run. The neural network is flexible, but the tree ensembles still separated safe and risky loans better on this tabular dataset.

Validation leaderboard



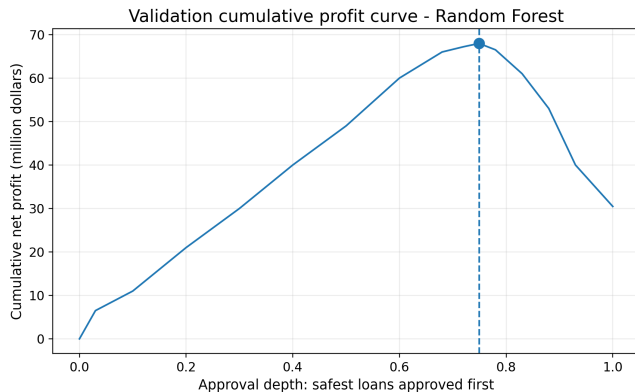
This leaderboard uses validation-tuned thresholds. Random Forest is currently the strongest model on the teaching sample.

What the leaderboard says

Model	AUC	Profit	Approval	Approved default
Random Forest	0.963	\$67.78M	74.8%	2.1%
Bagging	0.950	\$64.56M	70.0%	2.0%
Decision Tree	0.924	\$59.40M	69.4%	2.7%
Neural Net	0.902	\$58.50M	68.3%	3.9%
AdaBoost	0.905	\$55.09M	64.4%	3.0%
KNN	0.860	\$53.56M	64.1%	5.2%

Random Forest creates value by approving more good loans while keeping default leakage low.

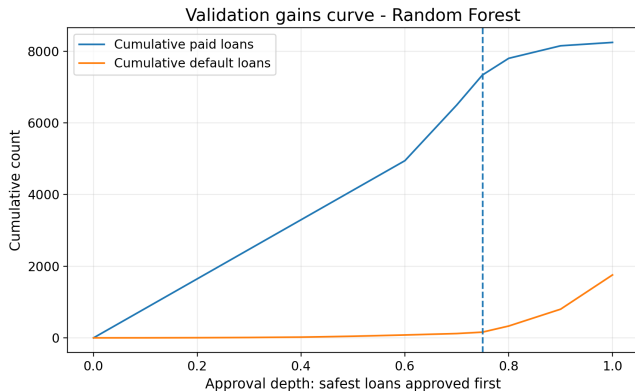
Profit curve: turn ranking into policy



Sort validation loans from safest to riskiest. Approve progressively and track cumulative profit. Best point in this run:

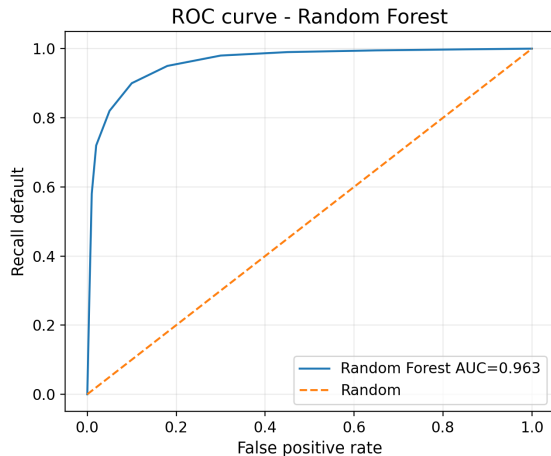
- approve about 74.9%
- default cutoff about 20.2%
- validation profit about \$67.97M

Gains curve: where defaults appear



Most paid loans appear early when we sort by predicted safety. Defaults stay low in the first part, then accumulate faster after the cutoff. This supports the approval policy: approve roughly the safest 75% and stop before the risk tail.

ROC, AUC, and Brier score



AUC measures ranking quality: can the model rank risky loans above safer loans?

Brier score measures probability error:

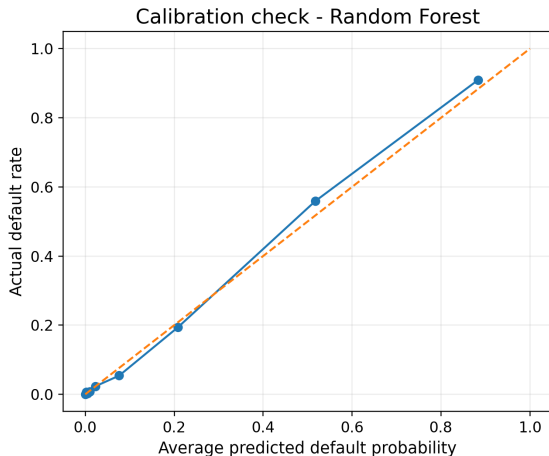
$$\text{Brier} = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i)^2$$

$\hat{p}_i$: predicted default probability.

y_i : actual result, 1 default and 0 paid.

Lower Brier is better.

Calibration check



Calibration asks whether predicted probabilities match actual default rates. Example: if a group of loans gets predicted around 20% default risk, about 20% of that group should default. The RF calibration is not perfect, but it is good enough to support a probability-based cutoff around 20%.

Error analysis at the chosen policy

Final validation policy:

approve if $P(\text{default}) \leq 0.2017$

Error type	Count	Profit effect
Correct approved paid loan	7,334	+\$79.22M
Correct denied default	1,597	\$0
Denied paid loan	910	\$0
Approved default	159	-\$11.25M

The remaining risk is concentrated in a small number of approved defaults with large average loan amounts.

Segment error rates

Raw error counts can be misleading because large states and sectors naturally have more loans. So I also checked error rates by segment.

Segment view	What to watch	Why
NAICS sector	dangerous error rate	industry risk differences
State / BankState	error rate, not just count	volume can dominate raw counts
RealEstate	approved-default rate	long-term loans behave differently
Recession	errors during exposed loans	regime effect
RevLineCr / LowDoc	documentation and credit-line risk	underwriting signal

Cross-validation result for Random Forest

max depth	min leaf	max features	CV AUC mean
16	50	None	0.9617
12	50	None	0.9615
16	50	0.5	0.9580
12	50	0.5	0.9577
12	50	sqrt	0.9143

This supports the tuning choice. In this encoded SBA dataset, sqrt feature sampling was too restrictive.

How the team should split the work

Person / task	Work
Everyone	Part 1 EDA, leakage check, feature ideas
Teammate 1	KNN and classification trees
Teammate 2	Logistic regression and discriminant analysis
Teammate 3	Neural network
Together	Compare results, profit curve, final test decision

Use the same leaderboard columns so we can compare results without arguing about different definitions.

Final test rule

Only run the final test after we freeze:

- predictor list
- feature engineering
- preprocessing choices
- model family and hyperparameters
- threshold rule

Then refit on train + validation, evaluate once on test, and stop tuning.

Main idea

Just keep the test set alone guys :(